# Python Virtual Environments, Demystified

*Or: why your venv broke when you uninstalled Python, and other revelations*

---

## The Mental Model Everyone Has (and why it's wrong)

Most people, when they first meet `python -m venv .venv`, build this mental model:

> A venv is a self-contained Python installation living inside my project folder.

This is **wrong**, but usefully wrong — it gets you through your first few projects. The truth is weirder and, once you see it, explains a huge amount of previously-baffling behavior.

## The Real Mental Model

> **A venv is a library folder plus a redirect sign.**

The `python.exe` (or `python` on macOS/Linux) inside `.venv/Scripts/` (or `.venv/bin/`) is not a Python interpreter. It's a tiny shim — roughly 100 KB — whose only job is to say:

1. "Go run the *real* Python over there at the path stored in `pyvenv.cfg`."
2. "But pretend `sys.prefix` is *here* and import packages from *my* `site-packages`."

That's it. That's the whole trick.

Open up `.venv/pyvenv.cfg` in any venv and you'll see something like:

```
home = C:\Python312
include-system-site-packages = false
version = 3.12.4
```

That `home` path is the forwarding address. The venv is essentially a P.O. box that forwards mail to wherever the real Python lives.

## The Interpreter/Library Split

Here's the key insight, and it's worth saying slowly:

- **Interpreter** → shared and borrowed from the base Python.
- **Libraries** → local to the venv and real.

When you `pip install numpy` inside an activated venv, that's not a fake install. Real wheels get downloaded, real C extensions get compiled, real files land in `.venv/Lib/site-packages/`. The

isolation you care about — "NumPy 1.26 in project A, NumPy 2.1 in project B, no conflict" — is real and rock-solid.

What's *not* duplicated is the interpreter itself. Which is why venvs are small and fast to create, and also why they're weirdly fragile.

## Consequences That Fall Out of This

Once you accept the shim-and-redirect model, a bunch of previously-mysterious behaviors suddenly make sense:

### 1. Uninstalling the base Python kills every venv built on it

The venv's forwarding address now points to a vacant lot. You get errors like:

```
did not find executable at 'C:\Python314\python.exe'
```

This bites macOS users constantly whenever Homebrew bumps Python from 3.11 to 3.12 — every venv built against the old version silently dies.

**Fix:** delete and recreate. Your `requirements.txt` or `pyproject.toml` is the actual source of truth, not the venv itself.

### 2. You can't move a venv to another machine

Or usually even another folder on the same machine. The paths baked into `pyvenv.cfg` and the activation scripts are absolute. This is why Docker images install dependencies fresh inside the container rather than copying a venv in from the host.

### 3. Venvs are cheap. Treat them as cattle, not pets

A fresh venv is ~15 MB, mostly pip's own dependencies. Nuking and recreating costs you nothing except one `pip install -r requirements.txt`. If something is weird, blow it away. Don't debug it.

### 4. `which python` lying is a feature, not a bug

When a venv is activated, `which python` (or `Get-Command python` in PowerShell) points to the shim. The shim redirects to the real interpreter. `sys.executable` inside Python reports the shim's path. Everyone agrees on the fiction, and the fiction is what gives you isolation.

## How This Compares to Other Tools

| Tool | Interpreter | Library isolation | Portable? | Speed |
|---|---|---|---|---|
| `venv` (stdlib) | Shim (borrowed) | Yes | No | Fast |

| Tool | Interpreter | Library isolation | Portable? | Speed |
|---|---|---|---|---|
| `virtualenv` | Shim (borrowed, copies a bit more) | Yes | No | Fast |
| `conda` | **Real interpreter** per env | Yes | More so | Slow |
| `uv` | Shim (borrowed) | Yes | No | Very fast |

A few notes on the table:

- **Conda is the odd one out.** Conda environments contain an actual full interpreter binary, which is why they're bigger, slower to create, and why conda can manage non-Python stuff like compilers, CUDA, and system libraries. It's a package manager masquerading as an environment manager, and vice versa.
- **uv** (the new Rust-based tool from Astral) uses the same shim trick as stdlib `venv` — it's just dramatically faster because it parallelizes the install step and has a smarter resolver.
- **virtualenv** predates the stdlib `venv` and still exists mostly for legacy reasons and support for older Python versions.

## The Debugging Checklist

When a venv misbehaves:

1. **Check `pyvenv.cfg`.** Does the `home` path still exist on disk?
2. **Check `python --version` inside the venv.** Does it match what you expect?
3. **Check `sys.executable` and `sys.prefix` in a REPL.** The first should point to the shim, the second to the venv root.
4. **Check `pip list`.** Are your installed packages actually there?
5. **When in doubt, nuke it.** `rm -rf .venv` and recreate. You have a `requirements.txt`, right? *Right?*

## The One-Sentence Summary

> A venv borrows an interpreter and owns its libraries — which is why it's cheap to create, fragile to its base Python, and impossible to relocate.

---

*If a package manager could sigh, `pip` would.*